How to Test a Next.js Project with Jest and Mocking

A practical PDF guide based on the saved explanation, covering setup, unit testing, React Testing Library, and common mocking patterns for Next.js projects.

What this guide covers
Setup with `next/jest`, component tests, mocked services, mocked `fetch`, mocked `next/navigation`, timers, project structure, and the limits of Jest with async Server Components.

Jest is commonly used in Next.js for unit tests and snapshot tests. With React Testing Library, it becomes a very practical choice for testing client components, hooks, helpers, and business logic. In real projects, the most valuable part is usually not only writing assertions, but learning how to replace outside dependencies with mocks so your tests stay fast, deterministic, and easy to reason about.

This guide keeps the explanation practical. Every section includes the purpose of the tool or pattern, followed by a working example.

1. Install the Testing Packages

In a typical Next.js project, start by installing Jest, the browser-like test environment, React Testing Library, and the matcher extensions used in most UI tests.

```
npm install -D jest jest-environment-jsdom @testing-library/react @testing-library/dom @t
```

Next.js provides built-in integration through `next/jest`. That integration helps align your tests with the project configuration and reduces boilerplate.

2. Create the Jest Configuration

Create a `jest.config.ts` file and use `next/jest` to generate the configuration. This lets Next.js handle the compiler, environment variables, stylesheet imports, and typical project defaults for test execution.

```
import type { Config } from 'jest'
import nextJest from 'next/jest.js'

const createJestConfig = nextJest({
  dir: './',
})

const config: Config = {
  coverageProvider: 'v8',
  testEnvironment: 'jsdom',
  setupFilesAfterEnv: ['<rootDir>/jest.setup.ts'],
}

export default createJestConfig(config)
```

3. Create the Setup File

The setup file is usually small. It extends Jest with browser-oriented matchers such as `toBeInTheDocument()`. These matchers make UI tests much more readable.

```
import '@testing-library/jest-dom'
```

4. Add Test Scripts to package.json

Add a normal test command, watch mode for local development, and a coverage command when you want a report.

```
{
  "scripts": {
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage"
  }
}
```

5. First Example: Testing a Client Component

The best place to start is a very small client component. This helps you understand rendering and assertions before you add data fetching or routing.

```
'use client'

type Props = {
  title: string
}

export default function GreetingCard({ title }: Props) {
  return <h1>{title}</h1>
}
```

```
import { render, screen } from '@testing-library/react'
import GreetingCard from './GreetingCard'

describe('GreetingCard', () => {
  it('renders the title', () => {
    render(<GreetingCard title="Hello Alfredo" />)
    expect(screen.getByText('Hello Alfredo')).toBeInTheDocument()
  })
})
```

This test renders the component, looks for the expected text in the screen, and asserts that it exists in the document. That is the core workflow for many interface tests.

6. How Mocking Works in Jest

Mocking means replacing a real dependency with a controlled fake. Instead of calling the real router, the real API, or the real timer, you substitute a test implementation so you can focus on one behavior at a time.

The most common tools are:

- `jest.fn()` for creating a mock function.
- `jest.mock()` for replacing an imported module.
- `jest.spyOn()` for observing or replacing a method on an existing object.
- `clearAllMocks`, `resetAllMocks`, and `restoreAllMocks` for cleanup between tests.

7. Mock a Simple Callback with jest.fn()

Use this when a component receives a function prop and you want to verify whether it was called and with which values.

```
import { jest } from '@jest/globals'

describe('mock function example', () => {
  it('tracks calls', () => {
    const onSave = jest.fn()

    onSave('course-1')
    onSave('course-2')

    expect(onSave).toHaveBeenCalledTimes(2)
    expect(onSave).toHaveBeenCalledWith('course-1')
    expect(onSave).toHaveBeenLastCalledWith('course-2')
  })
})
```

This is one of the cleanest patterns in unit testing because it isolates the interaction without needing a complete real implementation.

8. Mock a Helper or Service Module

A very common pattern in Next.js is separating data access into a service or helper file and mocking that module in your tests.

```
// lib/userService.ts
export async function getUserName() {
  return 'Real user'
}
```

```
'use client'

import { useEffect, useState } from 'react'
import { getUserName } from '@/lib/userService'

export default function UserInfo() {
  const [name, setName] = useState('')

  useEffect(() => {
    getUserName().then(setName)
  }, [])

  return <p>{name || 'Loading...'}</p>
}
```

```
import { render, screen, waitFor } from '@testing-library/react'
import UserInfo from './UserInfo'
import { getUserName } from '@/lib/userService'

jest.mock('@/lib/userService', () => ({
  getUserName: jest.fn(),
}))

describe('UserInfo', () => {
  it('renders mocked user name', async () => {
    ;(getUserName as jest.Mock).mockResolvedValue('Alfredo')

    render(<UserInfo />)

    await waitFor(() => {
      expect(screen.getByText('Alfredo')).toBeInTheDocument()
    })
  })
})
```

This approach is ideal when the component should be tested independently from the real API or database layer.

9. Mock fetch

If the component directly calls `fetch`, mock the global function to return a fake response. This prevents network calls during the test and lets you define the exact payload you want the component to receive.

```
'use client'

import { useEffect, useState } from 'react'

export default function Products() {
  const [items, setItems] = useState<string[]>([])

  useEffect(() => {
    async function load() {
      const res = await fetch('/api/products')
      const data = await res.json()
      setItems(data.items)
    }

    load()
  }, [])

  return (
    <ul>
      {items.map((item) => (
        <li key={item}>{item}</li>
      ))}
    </ul>
  )
}
```

```
import { render, screen, waitFor } from '@testing-library/react'
import Products from './Products'

describe('Products', () => {
  beforeEach(() => {
    global.fetch = jest.fn()
  })

  afterEach(() => {
    jest.clearAllMocks()
  })

  it('renders products from mocked fetch', async () => {
    ;(global.fetch as jest.Mock).mockResolvedValue({
      json: async () => ({
        items: ['Apple', 'Orange'],
      }),
    })

    render(<Products />)

    await waitFor(() => {
      expect(screen.getByText('Apple')).toBeInTheDocument()
      expect(screen.getByText('Orange')).toBeInTheDocument()
    })
  })
})
```

10. Mock next/navigation in the App Router

In App Router projects, routing is often the first Next.js-specific dependency you need to mock. A push function is usually enough for unit tests that only need to verify navigation.

```
'use client'

import { useRouter } from 'next/navigation'

export default function GoDashboardButton() {
  const router = useRouter()

  return (
    <button onClick={() => router.push('/dashboard')}>
      Go dashboard
    </button>
  )
}
```

```
import { fireEvent, render, screen } from '@testing-library/react'
import GoDashboardButton from './GoDashboardButton'

const pushMock = jest.fn()

jest.mock('next/navigation', () => ({
  useRouter: () => ({
    push: pushMock,
  }),
}))

describe('GoDashboardButton', () => {
  it('navigates to dashboard', () => {
    render(<GoDashboardButton />)

    fireEvent.click(screen.getByText('Go dashboard'))

    expect(pushMock).toHaveBeenCalledWith('/dashboard')
  })
})
```

The same idea can be extended to `usePathname` and `useSearchParams` when a component depends on them.

11. Mock next/image When Necessary

Some tests become simpler if `next/image` is replaced with a plain HTML image tag. This is not always necessary, but it is a common fallback when image behavior gets in the way of the specific assertion you want to make.

```
jest.mock('next/image', () => ({
  __esModule: true,
  default: (props: any) => {
    return <img {...props} />
  },
}))
```

12. Mock Timers

When a component depends on `setTimeout` or `setInterval`, fake timers let you move time forward inside the test instead of waiting in real time.

```
import { jest } from '@jest/globals'

beforeEach(() => {
  jest.useFakeTimers()
})

afterEach(() => {
  jest.useRealTimers()
})

it('waits 2 seconds', () => {
  const fn = jest.fn()

  setTimeout(fn, 2000)

  jest.advanceTimersByTime(2000)

  expect(fn).toHaveBeenCalled()
})
```

13. Suggested Project Structure

A simple structure keeps tests close to the files they verify and makes them easy to find during maintenance.

```
src/
  app/
  components/
    Button.tsx
    Button.test.tsx
  lib/
    api.ts
    api.test.ts
jest.config.ts
jest.setup.ts
```

A good rule is to test UI behavior in component tests, business logic in service or helper tests, and keep infrastructure calls mocked whenever possible.

14. Important Note About Async Server Components

Jest works well for client components, synchronous components, utility functions, and hooks. However, async Server Components are not currently a strong fit for Jest-based unit testing. Those flows are generally better covered with end-to-end testing tools.

15. TypeScript and ESM Caveat

If your setup uses ESM heavily, mocking behaves differently because imports are evaluated before the mock call. In those cases, you may need `jest.unstable_mockModule` together with dynamic imports.

```
import { jest } from '@jest/globals'

jest.unstable_mockModule('@/lib/api', () => ({
  getCourses: jest.fn(),
}))

const { getCourses } = await import('@/lib/api')
```

Many Next.js projects can stay with the simpler `jest.mock()` examples shown earlier, but it is useful to know this limitation exists.

16. Practical Recommendation

A strong practical stack for a Next.js project is:

- Jest + React Testing Library for components, hooks, helpers, and local business logic.
- Mocked services or mocked `fetch` for backend calls.
- Mocked `next/navigation` for routing behavior.
- End-to-end tests for authentication, real navigation, and async Server Component flows.

This combination usually gives the best balance between fast feedback and realistic confidence.

References

1. Next.js documentation: Testing with Jest.
2. Jest documentation: Mock Functions.
3. Jest documentation: The Jest Object and timer utilities.
4. Jest documentation: ECMAScript Modules.